Adsum

v 1.9.0

El primer lenguaje de programación multiidioma
con verificación formal de memoria

15 idiomas	4 targets	0 deps	Formal
humanos	arquitecturas	externas	verificado

Andrés Riascos

es · en · zh · hi · ar · bn · pt · ru · ja · id · fr · de · tr · ko · it

Confidencial

¿Qué es Adsum?

Un compilador de código nativo que acepta fuente en 15 idiomas humanos y genera binarios sin dependencias externas, con verificación formal de memoria en tiempo de compilación.

Multilingual

Escribe en tu idioma nativo — español, inglés, árabe, chino y 11 más

Formal

Lógica de separación + lifetimes verificados en compilación

Nativo

Sin runtime, sin GC, sin dependencias. Binarios estáticos puros

Multi-arch

x86_64 Linux · Windows PE32+ · ESP32-S3 · AArch64

Arquitectura del compilador

01	02	03	04	05	06
Traductor	**Parser**	**TypeChecker**	**FormalGuard**	**CodeGen**	**Linker**
15 idiomas → IR neutro	AST tipado	tipos + dominios	SepLogic + Lifetimes	emit_* por arquitectura	ELF / PE32+ / ELF32

Targets de compilación

ELF64	PE32+	ELF32 Xtensa	ELF64 AArch64
x86_64 Linux	x86_64 Windows	ESP32-S3 bare metal	AArch64 Linux

Mismo código fuente → cualquier target sin modificar el programa

Confidencial

Sistema de tipos y dominios semánticos

El compilador te permite correr algoritmos cuánticos como Qiskit, neuronas de computación neuromórfica (no con punto flotante), Blockchain definiéndolos como dominios de un mismo lenguaje.

SAFE_STACK

Variables locales, lógica de control

IO_VOLATILE

Red, JSON, stdin/stdout, sockets

CRYPTO_SECURE

Claves, firmas, buffers criptográficos

IOT_REALTIME

Reloj monotónico, deadlines, ZKP

QUANTUM

Registros y algoritmos cuánticos

IA_SENSORY

Redes neuronales, axones, dendritas

SeparationLogicVerifier garantiza en compilación que los dominios no se contaminan entre sí

Confidencial

Capacidades únicas en un solo lenguaje

Algoritmos cuánticos como builtins

grover · qft · shor · deutsch_jozsa · bernstein_vazirani

El compilador emite el circuito como código máquina nativo. Sin Qiskit.

Criptografía sin dependencias

SHA-256 · SHA-512 · ChaCha20 · HMAC-SHA256 · Ed25519 completo

Implementados como secuencias de procesador. Verificados contra vectores RFC.

Redes neuronales booleanas nativas

Axones de 28 bits · células de Schwann · dendritas con umbral

Inferencia con POPCNT — sin punto flotante, sin matrices.

Protocolo ERFR de red propio

TCP · UDP · HTTP con contexto de nodo

JSON, DNS y parsers de red emitidos como bytes de código máquina.

Confidencial

Adsum frente a otros lenguajes

Característica	Python	Rust	C	Java	Adsum
Compilación nativa AOT	✗	✓	✓	✗	✓
Sin GC	✗	✓	✓	✗	✓
Multiidioma humano	✗	✗	✗	✗	✓
Verificación formal	✗	Parcial	✗	✗	✓
Quantum como builtin	✗	✗	✗	✗	✓
Crypto sin librerías	✗	✗	✗	✗	✓

Confidencial

Adsum en práctica

El mismo programa — el mismo compilador — en cualquier idioma humano

#idioma: es

```
funcion principal(): Void {
  sea clave = "seed_hex"
  sea pk = generar_claves(clave)
  sea msg = "hola"
  sea firma = firmar(msg, clave)
  sea ok = verificar_firma(
    firma, msg, pk)
  escribir(ok)
}
```

=

#language: en

```
function main(): Void {
  let key = "seed_hex"
  let pk = keygen(key)
  let msg = "hello"
  let sig = sign(msg, key)
  let ok = verify_signature(
    sig, msg, pk)
  print(ok)
}
```

Ed25519 nativo · sin libsodium · verificado contra vectores RFC

Confidencial

Un mismo programa — todos los dominios

#idioma: es

```
funcion principal(): Void {

  // Circuito cuántico
  sea resultado = grover(8)
  sea fase = qft(4)

  // Red neuronal
  sea patron = axon_cargar(resultado)
  sea disparo = dendrita_disparar(
    patron, 0x7F, 3)

  // Ruta de red
  sea socket = udp_crear()
  sea msg = json_a_texto(disparo)
  udp_enviar(socket, msg,
    "192.168.1.1", 5000)
  retornar 0
}
```

QUANTUM

grover y qft emiten circuitos — nativo, cero librerías

IA_SENSORY

axones y dendritas con POPCNT — inferencia booleana pura

IO_VOLATILE

socket UDP + JSON builder — SepLogic verifica aislamiento de dominios

SeparationLogicVerifier garantiza en compilación que QUANTUM, IA_SENSORY e IO_VOLATILE no se contaminan

Confidencial

Adsum

El código en tu idioma.
La seguridad en el compilador.
El binario en tu hardware.

15 idiomas humanos	**4** targets en un mismo lenguaje
0 dependencias	**Formal** verificación

Andrés Riascos · v1.9.0

Confidencial